

Fine-Tuning de LLMs

Do modelo pré-treinado à adaptação eficiente

Ivina Lorena Oliveira Moura

Mestrado e Doutorado em Ciências da Computação

Universidade Federal do Ceará

Modelos de Linguagem de Grande Escala

Como adaptar um modelo e decidir quais parâmetros treinar?

Primeiro: o que é um modelo pré-treinado?

Uma tarefa repetida em muitos textos

“O aluno abriu o . . . ” → prever o próximo token.

- 1 O modelo atribui probabilidades às continuações possíveis.
- 2 O texto de treinamento fornece o próximo token observado.
- 3 O erro orienta mudanças nos pesos da rede.

Nosso ponto de partida

Pré-treinado: já passou por esse aprendizado.

Checkpoint: registro dos parâmetros aprendidos.

Fontes: Brown et al. (2020); transcrição MIT, abertura sobre modelos autoregressivos.

Por que ainda precisamos adaptar esse modelo?

Tarefa: encaminhar mensagens de clientes

Mensagem: “Não consigo entrar na minha conta.”

- Resposta plausível: “Tente redefinir sua senha.”
- Nossa necessidade: classificar a mensagem como **acesso**.
- Formato esperado: {"categoria": "acesso"}.

Pergunta

Produzir um texto plausível é o mesmo que cumprir essa tarefa?

Fontes: Ouyang et al. (2022), introdução e etapa SFT; transcrição MIT, trechos sobre instruções e fine-tuning. Exemplo sintético próprio.

Antes de treinar: onde está o problema?

Estratégia	O que muda	Uso a investigar
Prompting	Instruções e exemplos no contexto	Esclarecer a tarefa
RAG	Evidências recuperadas no contexto	Consultar fatos e documentos
Fine-tuning	Parâmetros persistentes	Aprender padrões de comportamento

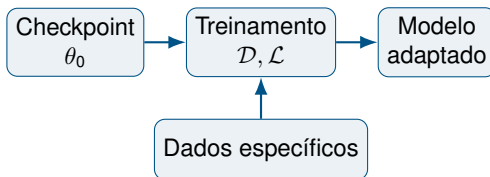
No caso dos chamados

Teste exemplos no prompt. Falhas persistentes podem motivar treinamento. Uma política interna ausente pode motivar recuperação.

As estratégias podem ser combinadas.

Fontes: Brown et al. (2020); Lewis et al. (2020); transcrição MIT, sequência RAG e fine-tuning.

Fine-tuning: continuar o aprendizado



O ciclo conhecido de redes neurais

Forward $\rightarrow$ loss $\rightarrow$ backward $\rightarrow$ atualização.

$$\theta_{k+1} = \theta_k - \eta \nabla_{\theta} \mathcal{L}_{\mathcal{B}}(\theta_k)$$

$\mathcal{B}$: minibatch; η : learning rate. SGD ilustrativo; Adam mantém estados adicionais.

Fontes: Transcrição MIT, trecho de fine-tuning; Kingma e Ba (2015), Adam.

Ensinar por demonstrações: Supervised Fine-Tuning

Uma demonstração

Entrada: “Não consigo entrar na conta.”

Resposta de referência: categoria **acesso**, no formato combinado.

SFT: ajuste fino supervisionado

Treinar com exemplos de entradas e respostas desejadas.

- Fine-tuning é o conceito geral de adaptação por treinamento.
- SFT especifica a supervisão: respostas de referência.
- Depois escolheremos quais parâmetros podem mudar.

Fontes: Ouyang et al. (2022), etapa SFT; Hu et al. (2022); Dettmers et al. (2023).

Um pequeno dataset de SFT

Instrução: classifique em acesso, cobrança ou falha;
responda apenas JSON.

Entrada	Resposta supervisionada
Não consigo entrar na conta.	<code>{"categoria": "acesso"}</code>
Recebi uma cobrança duplicada.	<code>{"categoria": "cobrança"}</code>
O aplicativo fecha ao abrir.	<code>{"categoria": "falha"}</code>

Antes do treino

Revisar rótulos; serializar papéis com o template do modelo; separar treino, validação e teste por origem do chamado.

Fontes: Exemplo próprio; Ouyang et al. (2022), demonstrações supervisionadas; transcrição MIT, instrução/entrada/saída.

Dos textos à loss: quais tokens contam?

Sequência didática	P_1	P_2	R_1	EOS
Máscara da loss	0	0	1	1

P : prompt; R : resposta. Segmentação e IDs reais dependem do tokenizer.

$$\mathcal{L}_{SFT} = -\frac{1}{\sum_t m_t} \sum_t m_t \log p_{\theta}(z_t | z_{<t})$$

- Labels são tokens-alvo, com deslocamento causal entre predição e alvo.
- Aqui, prompt e padding são ignorados na loss ($m_t = 0$).
- O prompt continua visível pela atenção.
- Teacher forcing: contexto anterior vem da sequência de referência.

Fontes: Objetivo autoregressivo: Brown et al. (2020); demonstrações SFT: Ouyang et al. (2022); esquema didático próprio.

Treinar bem exige mais do que baixar a loss

Controlar o treino

- Learning rate: passo.
- Epoch: passagem pelos dados.
- Batch: dados por atualização.
- Validação e checkpoint.

Avaliar a tarefa

- JSON válido.
- Macro-F1 de categorias.
- Novas origens e formulações.
- Regressão em tarefas gerais.

Fontes: Protocolo próprio; revisão arXiv:2408.13296v1, dados e avaliação; transcrição MIT.

Full Fine-Tuning: liberar toda a adaptação

$$\theta^* = \arg \min_{\theta} \mathcal{L}_{SFT}(\theta), \quad \theta \text{ iniciado em } \theta_0$$

- Todos ou praticamente todos os parâmetros são treináveis.
- A atualização não recebe uma restrição explícita de baixo rank.
- Há flexibilidade para mudanças amplas; qualidade superior não é garantida.
- Cada especialização pode exigir um checkpoint completo.

A conta

Pesos + gradientes + estados do otimizador + ativações.

Fontes: Hu et al. (2022), formulação do problema; Chen e Jia, slides 2–3; MIT, custos de fine-tuning.

Por que um modelo de 14 GB não treina em 14 GB?

Exemplo: $N = 7 \times 10^9$ parâmetros; GB decimais.

Componente assumido	Bytes/peso	Total
Pesos BF16	2	14 GB
Gradientes BF16	2	14 GB
Cópia mestre FP32	4	28 GB
Dois momentos Adam FP32	8	56 GB
Subtotal	16	112 GB

Ainda faltam ativações e buffers

Cenário ilustrativo, não exigência universal. Precisosões, sharding, offload e implementação mudam a conta.

Fontes: Cálculo próprio; Kingma e Ba (2015), dois momentos Adam; MIT, componentes de memória; Dettmers et al. (2023), §2.

O custo reaparece a cada especialização

Dez serviços, dez comportamentos

Dez checkpoints de 7B em BF16: $10 \times 14 = 140$ GB apenas em pesos.

- Forward e backward continuam atravessando a rede.
- Ativações dependem de batch, sequência e arquitetura.
- Checkpointing troca memória de ativações por recomputação.

Pergunta

Precisamos atualizar bilhões de parâmetros para cada comportamento?

Fontes: Contas próprias; Chen e Jia, custo e serving de adapters; Dettmers et al. (2023), memória e checkpointing.

PEFT: base preservada, adaptação pequena

Parameter-Efficient Fine-Tuning

$$\phi^* = \arg \min_{\phi} \mathcal{L}(f_{\theta_0, \phi}), \quad |\phi| \ll |\theta_0|$$

- θ_0 : base congelada nesta formulação.
- ϕ : poucos parâmetros acrescentados e treináveis.
- Outras variantes selecionam parte dos pesos existentes.

Economia com uma restrição

Menos estados treináveis e arquivos menores; a adaptação ocorre em um espaço mais limitado.

Fontes: Houlshby et al. (2019); Hu et al. (2022); Chen e Jia, panorama PEFT.

PEFT é uma família de escolhas

Família	O que se aprende	Onde atua
Soft prompts / prefixes	Vetores contínuos	Entrada / atenção
Adapters	Módulos adicionais	Entre transformações
LoRA	Fatores de uma atualização	Projeções lineares

Uma distinção importante

Escrever um prompt em texto não treina parâmetros. Soft prompt tuning envolve backpropagation.

Fontes: Chen e Jia, slides 5–16; Houlsby et al. (2019), adapters; Hu et al. (2022), trabalhos relacionados.

Congelar pesos não elimina o backward



- Não guardamos gradientes e estados Adam dos pesos congelados.
- O sinal precisa alcançar módulos treináveis anteriores.
- A base e parte das ativações continuam ocupando memória.

Pergunta

Como representar uma atualização com poucas variáveis?

Fontes: Houlsby et al. (2019); Dettmers et al. (2023), §2.

LoRA separa o conhecimento da mudança

Low-Rank Adaptation

$$W' = W_0 + \Delta W$$

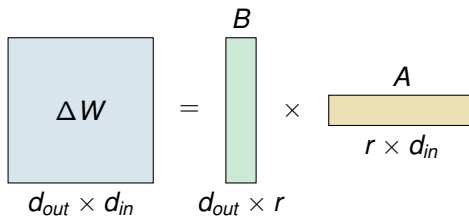
Ideia

Se uma atualização eficaz couber em poucas direções, podemos aprender uma representação pequena para ela.

- W_0 mantém a transformação pré-treinada.
- LoRA parametriza ΔW com fatores menores.
- Baixo rank é uma restrição a testar, não uma garantia para toda tarefa.

Fontes: Hu et al. (2022), §4 e §7; transcrição MIT, matrizes de mudança.

Visualizando a fatoração da atualização

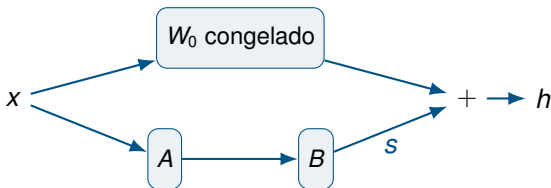


$$\Delta W = BA, \quad \text{rank}(BA) \leq r \ll \min(d_{in}, d_{out})$$

A projeta em r coordenadas; B combina essas coordenadas na saída. O produto pode ser denso.

Fontes: Hu et al. (2022), Fig. 1 e §4.1; derivação algébrica própria.

Duas rotas no forward; só uma é treinada



$$h = W_0x + sB(Ax), \quad s = \alpha/r$$

α : escala da formulação original. Inicialização usual: A aleatória e $B = 0$; inicialmente, $h = W_0x$.

Atualização

Treinamos A, B e mantemos W_0 fixo.

Fontes: Hu et al. (2022), §4.1; derivadas próprias pela regra da cadeia.

Quanto economizamos em uma projeção?

Exercício: $d_{in} = d_{out} = 4096$ e $r = 8$.

$$\underbrace{4096^2}_{\text{Full: 16 777 216}} \longrightarrow \underbrace{8(4096 + 4096)}_{\text{LoRA: 65 536}}$$

- Fração treinável: 0,390625%; redução de $256\times$.
- Fatores BF16: 128 KiB; base BF16: 32 MiB.

O denominador importa

É uma conta desta matriz, não uma redução de $256\times$ na VRAM total ou no tempo. A base continua presente.

Fontes: Cálculo próprio a partir da parametrização de Hu et al. (2022), §4.1.

Rank e módulos-alvo são decisões experimentais

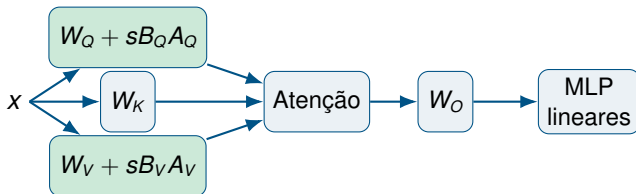
- Rank maior amplia capacidade e número de parâmetros.
- Atenção, MLP ou ambos: alvos mudam o espaço de adaptação.
- Comparar sob o mesmo protocolo de dados e avaliação.
- Mais rank não garante melhor generalização.

Por que LoRA pode funcionar?

A base oferece representações úteis; aprendemos correções estruturadas. A adequação depende da tarefa e do checkpoint.

Fontes: Hu et al. (2022), §7; Dettmers et al. (2023), análise dos módulos-alvo.

Onde a LoRA entra no Transformer?



Exemplo: adapters somente em Q e V . Todos os pesos da base ficam congelados; apenas os fatores A, B treinam.

Duas escolhas distintas

Módulos-alvo: onde adaptar. **Rank:** dimensão da correção em cada alvo.

Esquema simplificado: omite normalizações, conexões residuais e detalhes das cabeças. Projeções reais podem ter dimensões diferentes.

Fontes: Hu et al. (2022), §4.2 e §7.1; Dettmers et al. (2023), análise dos módulos-alvo. Exemplo didático próprio.

Da projeção ao modelo: quantos parâmetros treinam?

Hipóteses visíveis

32 camadas; 2 projeções 4096×4096 por camada; $r = 8$; somente fatores LoRA treináveis.

$$P_{\text{projeção}} = r(d_{\text{in}} + d_{\text{out}}) = 8(4096 + 4096) = 65\,536$$

$$P_{\text{camada}} = 2 \times 65\,536 = 131\,072$$

$$P_{\text{total}} = 32 \times 131\,072 = 4\,194\,304$$

- Fatores BF16: $4\,194\,304 \times 2 \text{ bytes} = 8 \text{ MiB}$.
- Se apenas r passar a 16: 8 388 608 parâmetros.

Conta própria de arquitetura hipotética. Não inclui base, gradientes, otimizador ou ativações; não é uma medição de VRAM.

Fontes: Derivação e números próprios a partir da parametrização de Hu et al. (2022), §4.1.

Mesmo número de parâmetros, escolhas diferentes

Mesmas 32 camadas; todas as projeções 4096×4096 .

Alvos por camada	Quantidade	Rank	Total treinável
Q, V	2	16	8 388 608
Q, K, V, O	4	8	8 388 608

O orçamento é igual; o espaço de adaptação muda

Maior rank em menos projeções ou menor rank em mais projeções. Nenhuma das escolhas garante melhor qualidade.

- Comparar na validação com protocolo e orçamento declarados.
- Registrar qualidade, tempo e pico de VRAM separadamente.

Exemplo hipotético: igualdade de parâmetros não prova equivalência funcional nem igualdade de custo total.

Fontes: Contas e comparação hipotéticas próprias; questão experimental motivada por Hu et al. (2022), §7.1, Tabela 5.

Depois do treino: adapter separado ou merge

$$W_{merge} = W_0 + sBA$$

- Base compartilhada + adapters: especializações compactas.
- Merge: uma matriz na inferência, sem ramo LoRA separado.
- Registrar base, tokenizer e configuração com o adapter.

Pergunta

Mesmo com poucos parâmetros treináveis, podemos reduzir a memória da base?

Fontes: Hu et al. (2022), §4.1; Chen e Jia, serving de adapters.

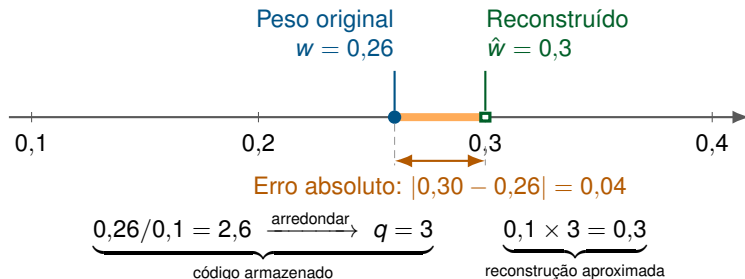
Quantização: armazenar aproximações com menos bits

Quantized Low-Rank Adaptation

Quantização uniforme para fins didáticos

$$s = 0,1$$

$$q = \text{clip}(\text{round}(w/s), q_{\min}, q_{\max}), \quad \hat{w} = sq$$



4 bits = 16 códigos. Os valores reconstruídos dependem do esquema e da escala do bloco; escalas também ocupam memória.

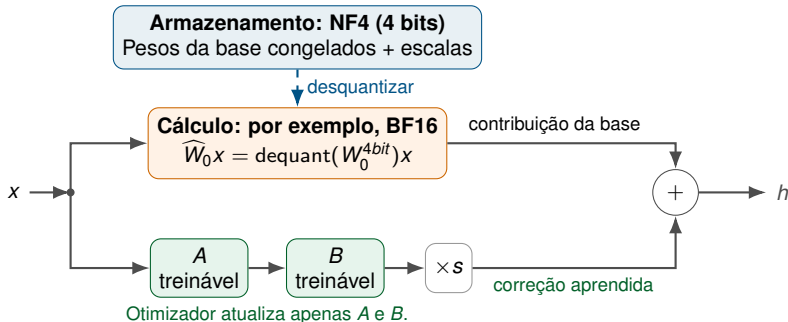
NF4 usa níveis não uniformes: esta reta não representa seu conjunto de valores.

Fontes: Reta e exemplo próprios; Dettmers et al. (2023), §2–3.

QLoRA: base em 4 bits, adapters treináveis

Quantized Low-Rank Adaptation

$$h = \text{dequant}(W_0^{4\text{bit}})x + sB(Ax)$$



Desquantização durante as operações: sem cópia BF16 permanente de toda a base. Buffers temporários ainda ocupam memória.

Aqui, $s = \alpha/r$ é a escala LoRA; não é a escala de quantização do slide anterior.

Double quantization: comprime também as constantes de quantização.

Paged optimizers: movimentam estados entre CPU/GPU para lidar com picos.

Fontes: Esquema próprio a partir de Dettmers et al. (2023), §3, eqs. (5)–(6).

Comparar memória exige separar componentes

Modelo ilustrativo de 7B; mesma tarefa e sequência.

Componente	Full	LoRA	QLoRA
Base armazenada	14 GB	14 GB	3,5 GB*
Estados treináveis	$O(N)$	$O(P)$	$O(P)$
Ativações e buffers	Sim	Sim	Sim

* Apenas $7 \times 10^9 \times 4/8$ bytes; exclui metadados e módulos não quantizados.
Full/LoRA: base BF16. P : parâmetros dos adapters.

Peso armazenado não é pico de VRAM

Contexto, batch, checkpointing, kernels e otimizador mudam o pico e o tempo.

Fontes: Contas próprias; Dettmers et al. (2023), §2–3 e apêndice G.

QLoRA: memória menor, qualidade preservada?

Viabilidade publicada

Adaptação de uma base de 65B em uma GPU de 48 GB, na configuração do artigo.

Tabela 4: acurácia MMLU 5-shot (%).

Método	7B / FLAN v2	65B / FLAN v2	Média*
LoRA BF16	45,6	62,5	53,0
QLoRA FP4	44,0	63,3	52,2
QLoRA NF4 + DQ	44,5	63,9	53,1

* Média das oito configurações da tabela original: quatro tamanhos, dois datasets. Não é a média das duas colunas exibidas.

Leitura correta

Comparação com **LoRA BF16**, não Full FT. Média próxima não garante igualdade em cada configuração ou tarefa.

Fontes: Tabela adaptada de Dettmers et al. (2023), arXiv:2305.14314v1, Tabela 4, p. 8.

Full, LoRA ou QLoRA: o que estamos trocando?

Método	Atualizados	Memória	Custo	Flexibilidade	Cenário típico
Full	Toda / quase toda base	Estados da base + ativações	Mais estados e checkpoints grandes	Sem restrição de rank	Recursos amplos; mudança extensa
LoRA	Fatores A, B	Base + estados pequenos + ativações	Menos estados; medir tempo	Rank e alvos limitam	Base cabe; muitas especializações
QLoRA	Fatores A, B	Base comprimida + estados + ativações	Menos VRAM; conversão tem custo	LoRA + aproximação	Base limita a VRAM

Mesma base e tarefa. Não há vencedor universal em qualidade ou tempo.

Fontes: Síntese própria de Hu et al. (2022), §4, e Dettmers et al. (2023), §2–4; Chen e Jia.

A loss melhora, mas a tarefa continua falhando

Caso hipotético: classificação de chamados

A loss de treino cai. O modelo passa a produzir JSON válido, mas continua confundindo cobrança com acesso.

Hipótese	Evidência a examinar
Aprendeu mais o formato que a categoria	Validade do JSON e erros por classe
Há rótulos ambíguos ou inconsistentes	Revisão de exemplos e política de anotação
O ajuste não generaliza	Curvas de validação, checkpoints e casos novos

Próxima decisão

Comparar com o modelo-base; selecionar mudanças na validação. Uma loss menor, sozinha, não identifica a causa.

Fontes: Caso e protocolo próprios; avaliação de tarefa discutida em Ouyang et al. (2022) e Dettmers et al. (2023). Nenhum número experimental é atribuído a esses artigos.

A tarefa nova melhora, mas outra capacidade piora

Caso hipotético

Após SFT para classificar chamados, o modelo piora ao resumir mensagens, capacidade também necessária na aplicação.

- 1 Comparar antes/depois com os mesmos casos e protocolo.
- 2 Verificar prompt, geração, tokenizer e versão do adapter.
- 3 Examinar composição dos dados e sequência do treinamento.
- 4 Avaliar qualidade nova e retenção da capacidade anterior.

Regressão observada não determina sua causa

Esquecimento é uma hipótese possível. A avaliação deve distinguir perda de capacidade de mudança de comportamento ou configuração.

Fontes: Dong et al. (2024), How Abilities in Large Language Models are Affected by Supervised Fine-tuning Data Composition, ACL, <https://aclanthology.org/2024.acl-long.12/>. Caso de chamados e protocolo: elaboração própria.

Como comparar as adaptações de forma defensável?

Condição	Pergunta experimental
Base com prompting	Qual é o desempenho antes de treinar?
SFT com LoRA / BF16	Quanto os adapters ajudam nesta tarefa?
SFT com QLoRA	Qualidade, memória e tempo mudam como?
Full SFT, se viável	O que muda ao ampliar os pesos treináveis?

Duas comparações possíveis

Efeito de uma escolha: controlar as outras condições.

Melhor solução por orçamento: ajustar cada método na validação, com orçamento declarado.

Mesma origem da base, dados e critérios comparáveis. Registrar configuração, seeds e hardware; medir qualidade, validade, regressões, VRAM e tempo. Teste final reservado.

Fontes: Protocolo proposto pela docente; fundamentação dos métodos: Hu et al. (2022), Dettmers et al. (2023) e Dong et al. (2024). Não há resultados medidos neste quadro.

Fechamento: começar pelo erro observado

- 1 Identificar o problema: instrução, informação ou comportamento.
- 2 Se for necessário treinar, usar demonstrações para orientar o SFT.
- 3 Escolher a adaptação: Full atualiza a base; LoRA aprende correções; QLoRA comprime também a base.
- 4 Avaliar qualidade na tarefa, memória e tempo de execução.

Mensagem final

A técnica deve responder ao problema e aos recursos disponíveis; a avaliação mostra se a adaptação funcionou.

Fontes: Síntese própria dos materiais científicos e didáticos listados na aula.

Referências principais

- Hu et al. (2022). *LoRA: Low-Rank Adaptation of Large Language Models*. ICLR. arXiv:2106.09685.
- Dettmers et al. (2023). *QLoRA: Efficient Finetuning of Quantized LLMs*. NeurIPS. arXiv:2305.14314.
- Ouyang et al. (2022). *Training language models to follow instructions with human feedback*. NeurIPS. arXiv:2203.02155.
- Brown et al. (2020). *Language Models are Few-Shot Learners*. NeurIPS. arXiv:2005.14165.
- Houshy et al. (2019). *Parameter-Efficient Transfer Learning for NLP*. ICML. PMLR 97:2790–2799.

Referências e materiais de apoio

- Lewis et al. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. NeurIPS. arXiv:2005.11401.
- Kingma e Ba (2015). *Adam: A Method for Stochastic Optimization*. ICLR. arXiv:1412.6980.
- *The Ultimate Guide to Fine-Tuning LLMs from Basics to Breakthroughs*, v1.0 (2024). arXiv:2408.13296v1. Revisão de apoio.
- Chen e Jia. *LLM Finetuning Techniques*. 15-779, Lecture 17 (2025). PDF local.
- Ramakrishnan. *Generative AI – Adapting LLMs with Parameter-Efficient Fine-Tuning*. MIT OCW, 15.773, aula 10 (2024). Transcrição local.
- Dong et al. (2024). *How Abilities in Large Language Models are Affected by Supervised Fine-tuning Data Composition*. ACL. ACL Anthology: 2024.acl-long.12.

Inventário e limitações de acesso: `sources/fontes_para_aula.md`.